

## FORME — Complete UI Design & Function Breakdown

The generated image is a visual blueprint for your FORME e-commerce website. It contains six design areas: five desktop page layouts and three mobile screens. The purpose is to use one consistent visual system across every page while giving each page a different function.

I'll break down the image section by section, explain every major element, describe how it should behave, and show how to map it to your Next.js project.

### 1. Overall design system



FORME is a fictional premium home and lifestyle brand selling objects such as table lamps, desk accessories, bags, and decorative items. The design language is based on five principles:

### 1. Warm neutral color palette

Ivory backgrounds, charcoal text, subtle gray borders, and muted brown accents create a consistent identity. The product photography supplies most of the visual color.

### 2. Editorial typography

Large serif headlines communicate the brand's personality. Sans-serif text is used for navigation, product details, filters, prices, and buttons because these elements need to be easy to scan.

### 3. Consistent spacing

Each page uses predictable horizontal margins, gaps between cards, and vertical spacing between sections. This makes the site feel like one product.

### 4. Product-first photography

Large, carefully cropped images communicate material, shape, and texture. Product images should have consistent aspect ratios and avoid looking randomly arranged.

### 5. Functional minimalism

Buttons, icons, filters, and product controls should have clear purposes. Decorative elements should not distract from browsing and purchasing.

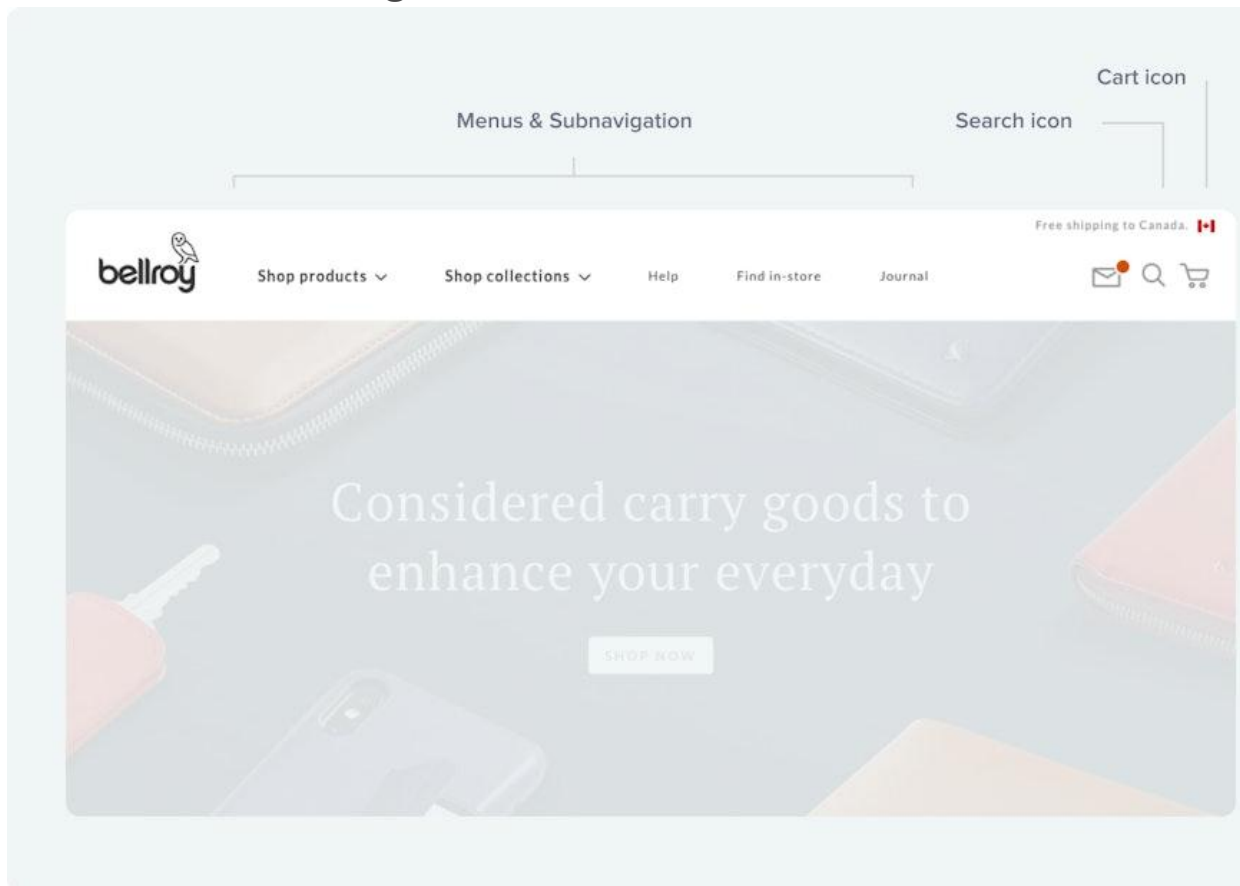
## Recommended global design tokens

| Token           | Suggested value | Function                              |
|-----------------|-----------------|---------------------------------------|
| Page background | #F7F5F0         | Creates the warm ivory canvas         |
| Primary text    | #252522         | Headings, product names, buttons      |
| Secondary text  | #77746D         | Descriptions, breadcrumbs, metadata   |
| Borders         | #E5E1D8         | Separators, inputs, card outlines     |
| Accent          | #B68C63         | Selected options and small highlights |
| Primary button  | #252522         | Main purchase and navigation actions  |
| Card radius     | 2–6px           | Restrained, editorial styling         |

|               |               |                                |
|---------------|---------------|--------------------------------|
| Content width | 1,200–1,300px | Keeps desktop content readable |
|---------------|---------------|--------------------------------|

These are proposed implementation values inspired by the reference, not measurements extracted from the generated image.

## 2. The shared navigation bar



The navigation bar appears at the top of every desktop screen and at the top of each mobile screen. It is the most important shared component because inconsistent navigation was the problem you identified earlier.

### Element-by-element breakdown

FORME logo

Position: far left

Returns visitors to the homepage. It should use the same font treatment, size, alignment, and spacing on every route.

Shop

Position: center-left

Links to /products, where customers can browse the full catalog.

## Collections

Links to `/collections/lighting`, `/collections/desk-accessories`, and other categories. You can implement this as a dropdown on desktop and an expandable menu on mobile.

## About

Introduces the fictional brand, its design philosophy, and the products' materials. You can add an About page after the core shopping flow is finished.

## Search icon

Opens a search input or search overlay. Users can search by product name or category. Include an empty state when there are no matches.

## Account icon

Can link to an account page or sign-in screen. Since authentication isn't essential to your frontend portfolio, this can be a secondary feature.

## Shopping bag icon

Opens `/cart`. A small badge should display the total number of items in the cart and update whenever an item is added or removed.

## Header implementation

File: `src/components/layout/Header.tsx`

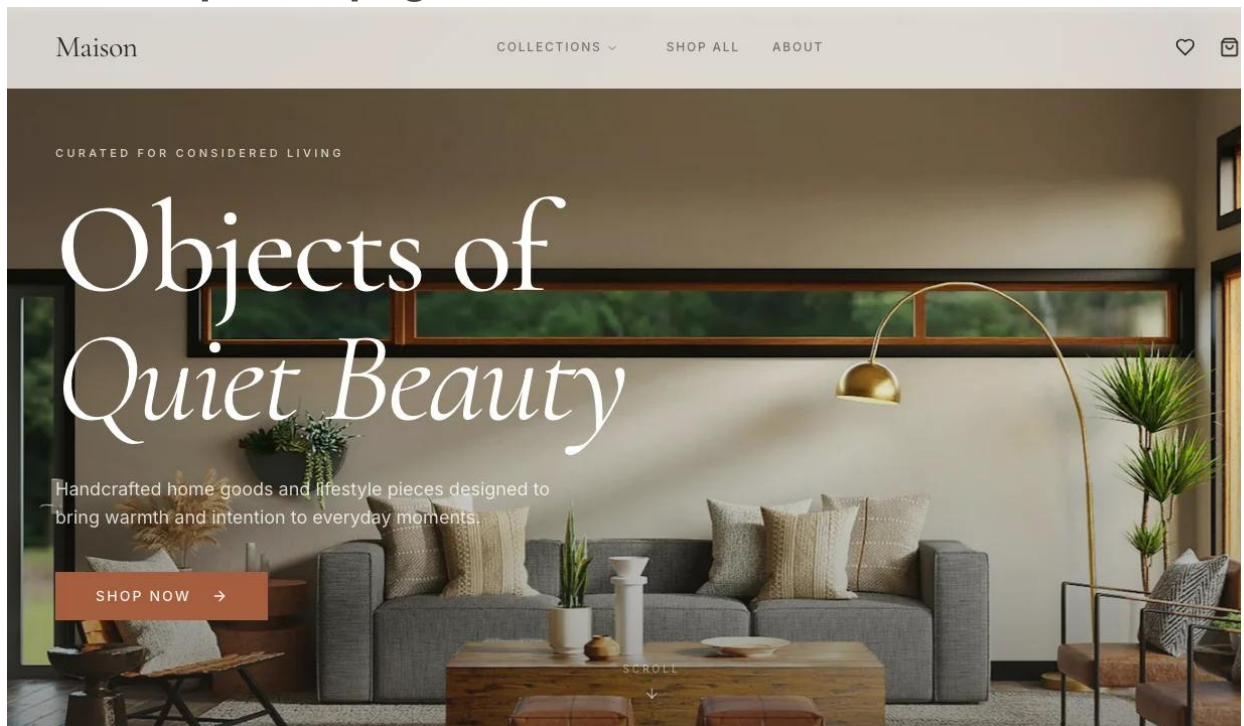
The header should be implemented once inside `src/app/layout.tsx`, rather than copied into every page.

Its behavior should be consistent:

- The logo always links to `/`.
- Shop always links to `/products`.
- The cart count reads from shared cart state.
- Mobile navigation collapses into a menu.
- All interactive elements support keyboard navigation and visible focus

states.

### 3. Desktop homepage — the brand introduction



Route: /

Purpose: Introduce the brand, create visual interest, and direct visitors toward the product catalog.

The homepage in the generated image has four major sections.

#### A. Hero section

##### Layout



Left side: text and call to action

- Main headline: "Thoughtful objects for everyday living"
- Supporting copy describing the brand
- Dark button: "Shop the collection"

Right side: hero image

- Large table lamp
- Books and a small decorative object
- A branch or plant for visual balance

The split composition creates a clear reading order: headline first, product image second, action button next.

Function: Clicking the CTA takes users to /products or a selected collection.

Frontend details:

- Use CSS Grid or Flexbox for the text/image layout.
- Stack the content vertically on narrow screens.
- Use next/image for optimized product photography.
- Avoid text baked into the hero image; keep the headline and CTA as actual HTML.

## **B. Featured collections**

The next section contains four image cards representing categories:

- Lighting

- Desk Accessories
- Bags
- Home Essentials

Each card includes an image, a category name, and potentially a small arrow.

Function: Clicking a card opens the relevant collection.

For example:

/collections/lighting

All four cards should share a reusable `CollectionCard` component with the same image proportions, text placement, and hover behavior.

### **C. Brand story**

This section pairs an editorial image of a material or interior detail with a short brand statement.

Its function is not to sell a specific product immediately. It gives the fictional brand a personality and makes the storefront feel more complete.

Use a two-column layout on desktop and stack the image and text on mobile.

### **D. Newsletter and footer**

The newsletter section contains an email field and a `Subscribe` button. For the portfolio version, you can implement client-side email validation and a simulated success state rather than setting up a mailing-list service.

The footer contains links such as `Shop`, `About`, `Journal`, and `Contact`, alongside social icons.

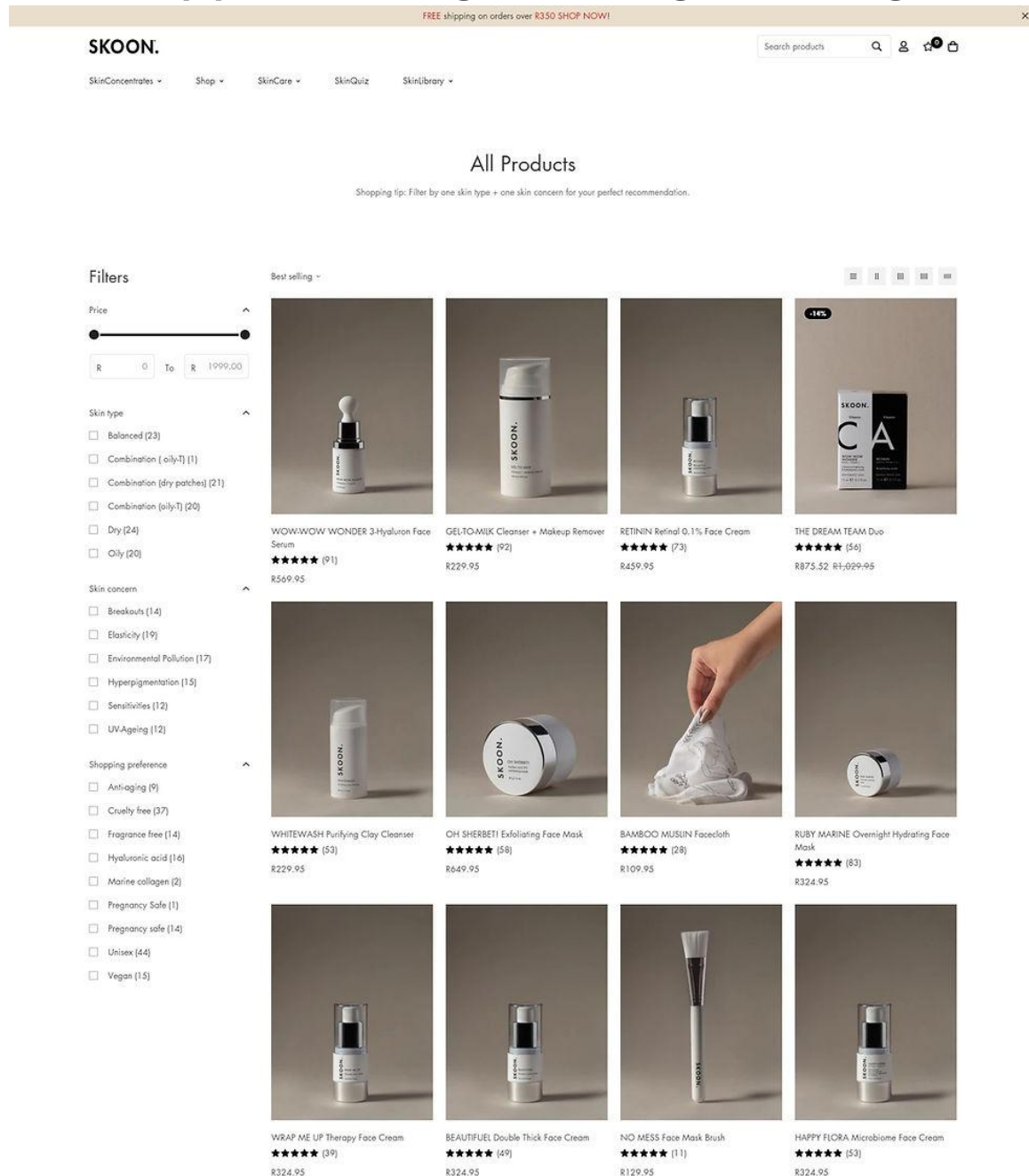
Important: Only make these links clickable if they have real destinations or a deliberate placeholder page.

### **Homepage component structure**

```
app/page.tsx
├── HomePage
│   ├── Hero
│   ├── FeaturedCollections
│   │   └── CollectionCard
│   ├── FeaturedProducts
│   │   └── ProductCard
│   └── BrandStory
```

The Footer should actually be rendered by the root layout, not duplicated inside HomePage. The diagram above simply shows where it appears visually.

## 4. Desktop product listing — browsing the catalog



Route: /products

Purpose: Help users find a product among multiple items.

The image shows a left filter sidebar and a larger product grid on the right.



## A. Page heading

At the top:

- Title: "All Products"
- Short descriptive subtitle
- Search field
- Sort dropdown

These elements tell the user where they are and provide immediate access to browsing controls.

## B. Filter sidebar

The sidebar is divided into three groups.

### Category filter

Example options shown in the design

All

Lighting

Desk Accessories

Bags

Home Essentials

### Price filter

Under \$50

\$50–\$100

\$100–\$200

\$200+

### Color filter

Beige

Black

Brown

White

These are illustrative controls. In the actual website, each filter must change the products displayed.

Example: Selecting "Lighting" should show lamps and other lighting products, not simply highlight the checkbox.

The filtering logic should support combinations. If a user selects Lighting and a price range, the result must satisfy both conditions.

## C. Product grid

Each card displays:

- Product image
- Optional wishlist button
- Product name
- Price
- Optional hover effect or quick-view action

Example catalog items:

| Product            | Example price |
|--------------------|---------------|
| Luna Table Lamp    | \$129         |
| Canvas Tote Bag    | \$89          |
| Ceramic Pen Holder | \$45          |
| Oak Desk Organizer | \$79          |
| Minimal Wall Clock | \$99          |

These are fictional example prices. Choose one currency for the real implementation; NT\$ is a natural choice if you want the site to feel targeted to Taiwanese users.

### Product card behavior

When the user clicks the product image or title, navigate to `/products/luna-table-lamp`.

The wishlist icon should toggle its state if implemented. A hover effect should not be the only way to discover an important action, because mobile devices do not have conventional hover behavior.

## D. Sorting and pagination

The sort dropdown could provide:

- Featured
- Price: low to high
- Price: high to low

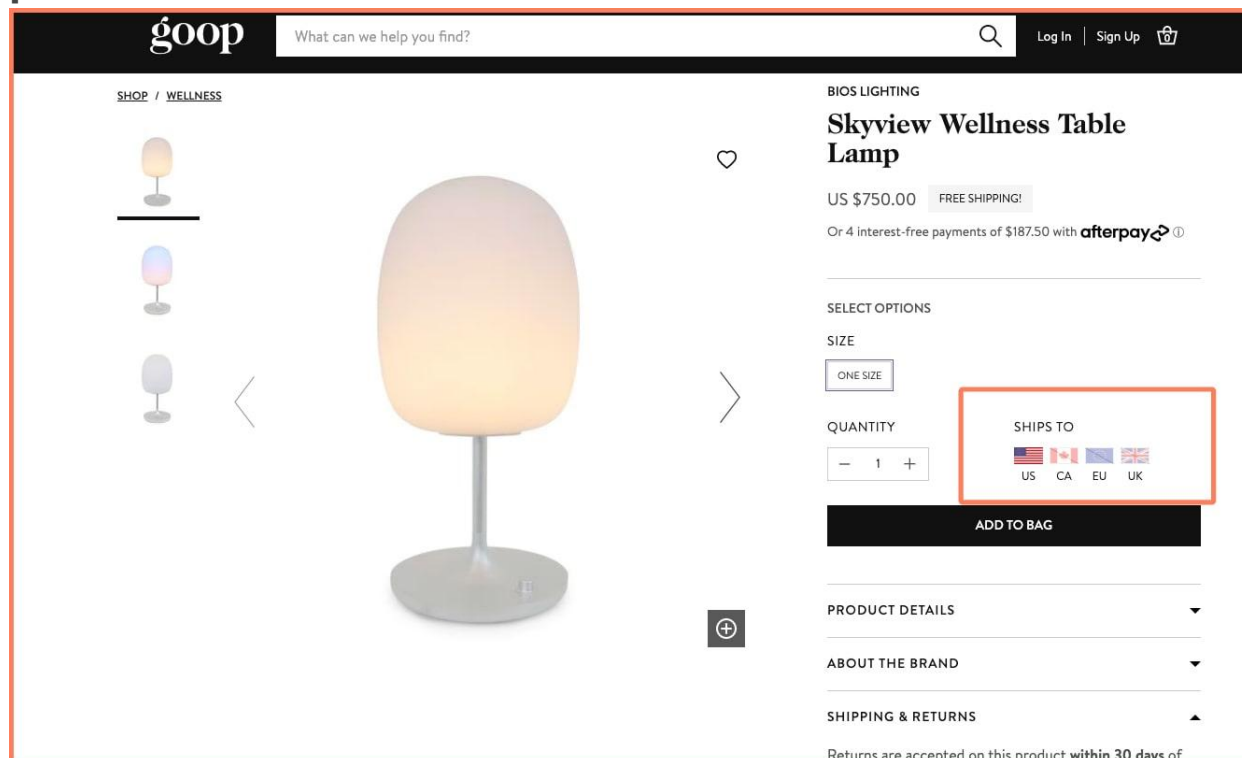
- Newest

Pagination or a "Load more" button can appear below the grid.  
For a small catalog, showing all products initially is fine. Pagination becomes useful once the number of items grows.

## Product listing component structure

```
app/products/page.tsx
├── ProductListingPage
│   ├── PageHeading
│   ├── ProductSearch
│   ├── ProductFilters
│   ├── SortSelect
│   ├── ActiveFilterChips
│   ├── ProductGrid
│   │   └── ProductCard
│   └── Pagination
```

## 5. Desktop product detail — turning interest into a purchase



Route: /products/[slug]  
Example: /products/luna-table-lamp

Purpose: Give the visitor enough information to decide whether to add the item to their cart.

### **A. Breadcrumb navigation**

The top of the page displays something similar to:

Home / Lighting / Luna Table Lamp

This tells users where they are in the site hierarchy.

Each breadcrumb except the current product should link to the corresponding page.

### **B. Product gallery**

The left side of the desktop layout contains:

- A vertical strip of thumbnail images
- One large primary image
- An optional favorite button

Clicking a thumbnail changes the primary image.

For your implementation, store several image paths for each product rather than creating a separate page for every photo.

### **C. Product information**

The right side contains:

- Product name
- Price
- Optional rating and review count
- Product description
- Color or size selector
- Quantity selector
- Add-to-cart button

- Shipping and returns information

The visual hierarchy should lead the visitor from the product name to the price, options, and primary action.

### **Variant selector**

The generated image shows circular color swatches.

Clicking a swatch should update the selected variant. If different variants have different images or stock levels, those values should update as well. A selected swatch needs a clear visual indicator and an accessible label such as "Oak finish."

### **Quantity selector**

The minus and plus buttons change the requested quantity.

Rules to implement:

- Quantity cannot fall below one.
- Adding an item should respect any available stock limit.
- The cart should receive the selected quantity and variant.

### **Add to cart**

This is the main action on the page.

After clicking:

1. Add the selected product and variant to shared cart state.
2. Update the header's cart count.
3. Show clear feedback, such as "Added to bag."
4. Allow the user to continue shopping or navigate to the cart.

The button should also handle rapid repeated clicks without producing inconsistent cart quantities.

## **D. Product details and recommendations**

Below the main section, the page includes description tabs or expandable

panels, followed by related products.

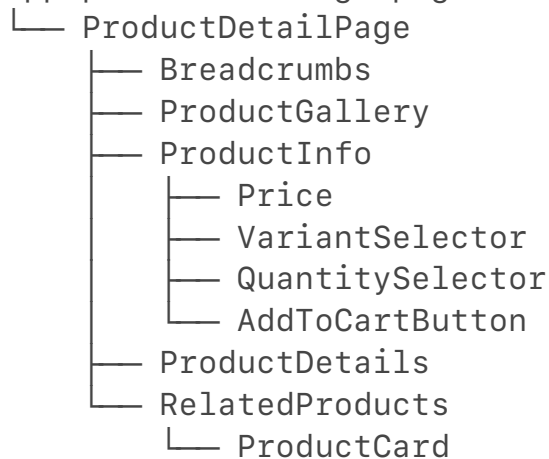
Possible sections:

- Description
- Materials and dimensions
- Shipping and returns
- You might also like

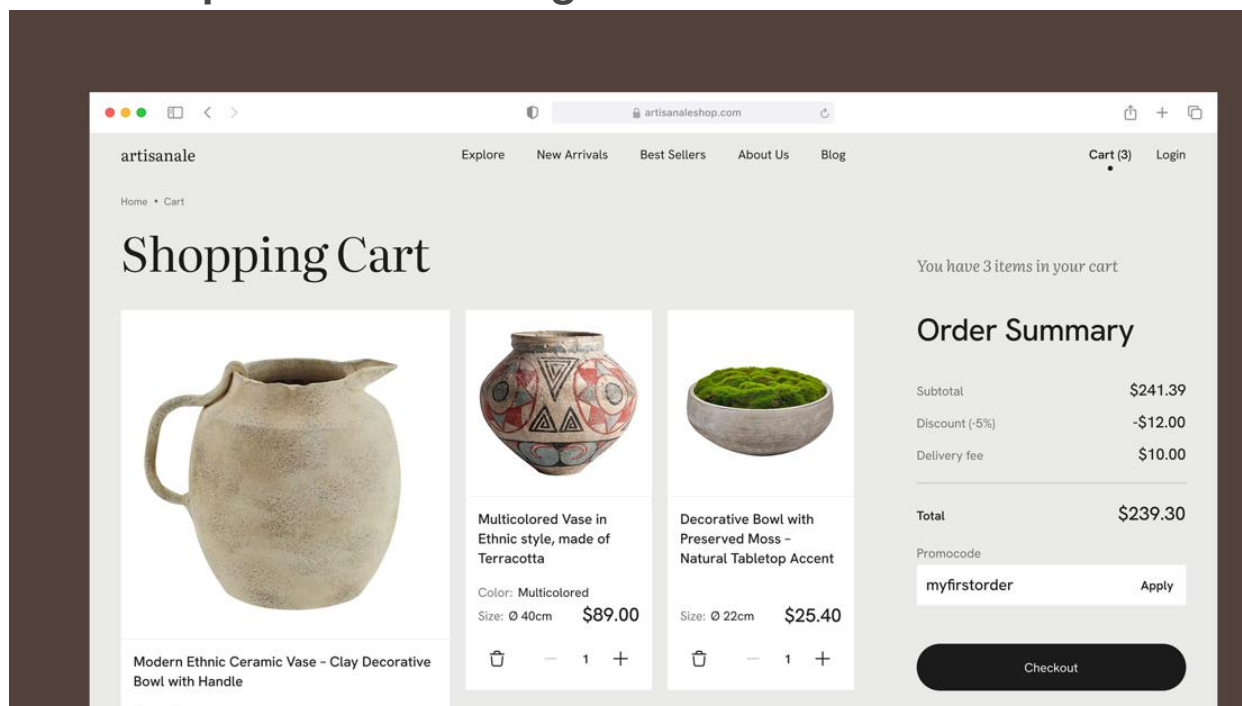
Related products should link to their own product detail pages.

## Product detail component structure

app/products/[slug]/page.tsx



## 6. Desktop cart — reviewing the order



Route: /cart

Purpose: Let customers review and edit their selected products before checkout.

The generated image has two main columns.

### **A. Cart item list**

Each row contains:

- Product thumbnail
- Product name
- Selected variant
- Unit price
- Quantity controls
- Remove button

For example, the cart may contain a Luna Table Lamp and a Canvas Tote Bag.

If the user increases the quantity of the lamp from one to two, the subtotal must immediately reflect the new amount.

If they remove the tote bag, the cart item list and totals must update together.

### **B. Order summary**

The right-hand panel contains:

- Subtotal
- Shipping cost
- Total
- Proceed to checkout button

A basic calculation is:

Subtotal

=

$\sum$

$i$

=

1

$n$

(

price

$i$

$\times$

quantity

$i$

)

$\text{Subtotal} = \sum_{i=1}^n (\text{price}_i \times \text{quantity}_i)$

$\text{Subtotal} = \sum_{i=1}^n (\text{price}_i \times \text{quantity}_i)$

Then:

Total

=

Subtotal

+

Shipping

+

Tax

$\text{Total} = \text{Subtotal} + \text{Shipping} + \text{Tax}$

$\text{Total} = \text{Subtotal} + \text{Shipping} + \text{Tax}$

For the initial portfolio version, you can simplify this to subtotal plus shipping and explicitly define the shipping rule. Don't add tax calculations unless you implement a consistent tax model.

### C. Empty-cart state

You also need a design for when the cart has no products.

It should display a message such as "Your bag is empty," a brief explanation, and a "Continue shopping" button.

This is important because a cart is not always populated.

### D. Cart persistence

Use shared React state and browser storage so the cart survives page refreshes.

A useful architecture is:

- CartContext.tsx: shared cart state and operations.



- useCart.ts: convenient access to cart functionality.
- cart-storage.ts: safely reads and writes cart data.
- CartSummary.tsx: computes and displays totals.

You do not need a database for this initial implementation.

## 7. Desktop checkout — completing the flow

**Fastbike** Secure Checkout [Return to Shopping](#)

1. Address 2. Shipping 3. Payment 4. Order Confirmation

**Buying As**

☒ Private Person ☐ Company

**Shipping Address**

First Name  Last Name

Street and House Number

Zip Code  City

Country

Phone Number

By providing your mobile number, you're agreeing to receive SMS updates regarding your order. Standard rates apply.

☒ I allow Fastbike to use the content of my basket and my email address to send me reminders regarding my abandoned baskets.

**Billing Address**

☒ Billing address is same as shipping address

[Continue to Shipping](#)

**Order Summary**

|                     |                   |
|---------------------|-------------------|
| Subtotal            | 11 478.90€        |
| Shipping (Standard) | 59.90€            |
| <b>Total</b>        | <b>11 538.80€</b> |

Including 24% VAT (Estonia)

[Chat with Us](#)

**Delivery within 3-5 business days**

**Speedmax CFR TT** 11 459.00 €

Color: Pro Black  
Frame Size: S  
Quantity: 1

Fastbike © 2025 - All Rights Reserved [Imprint](#) [Terms & Conditions](#) [Privacy Policy](#)

Route: /checkout

Purpose: Collect the information needed for a simulated order and show users a clear summary before completion.

The generated design uses a multi-step checkout indicator:

1

Information

Email and delivery details

2

Shipping

Delivery method and address confirmation

3

Payment

Simulated payment and order confirmation

## **Form fields**

The first step includes fields such as:

- Email address
- Full name
- Street address
- City
- Postal code

Each field needs a visible label, a useful input type, and validation feedback.

Example: If the email field is invalid, display an inline message near the field instead of showing only a generic alert.

## **Order summary sidebar**

The summary should display the products, quantities, subtotal, shipping cost, and final total.

It should use the same calculation logic as the cart page. This prevents the cart and checkout from displaying different prices.

## **Payment and confirmation**

For your portfolio, do not collect real card details or process real payments. Use a clearly simulated payment step and display a confirmation page when the test order succeeds.

The confirmation page could show:

- A success message

- A mock order number
- The purchased items
- The final total
- A button to return to shopping

## Checkout component structure

```

app/checkout/page.tsx
├── CheckoutPage
│   ├── CheckoutSteps
│   ├── ContactForm
│   ├── ShippingForm
│   ├── SimulatedPayment
│   └── OrderSummary

```

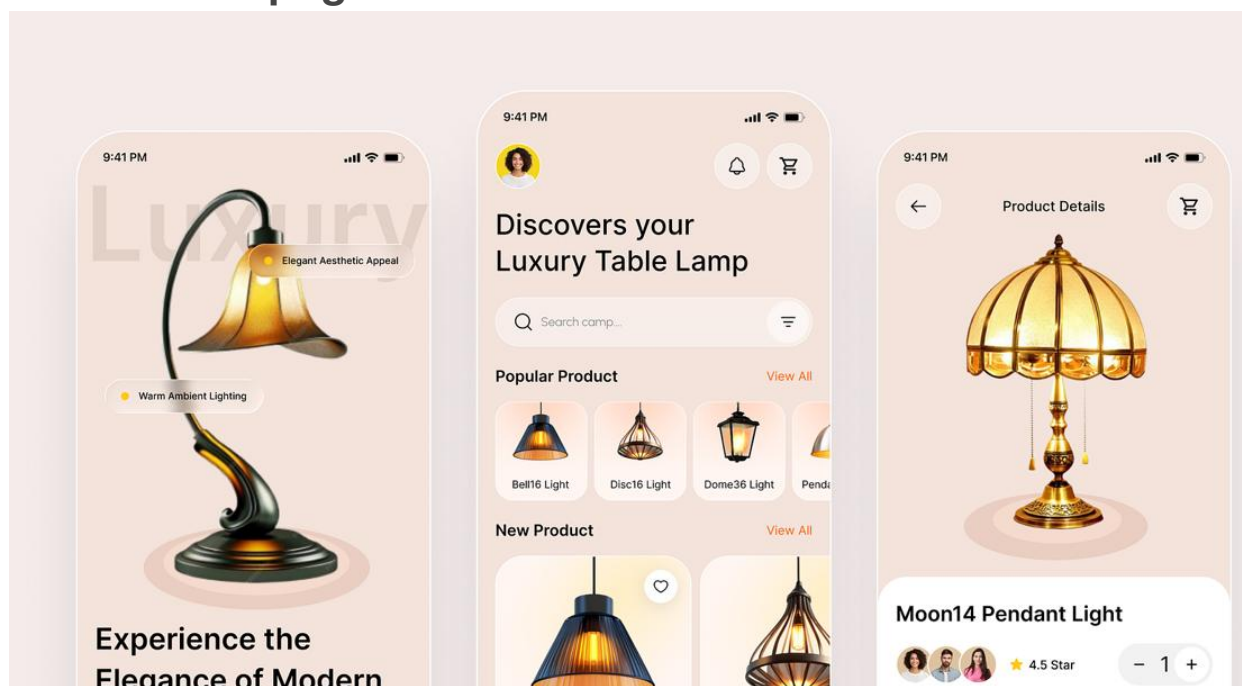
The success page should live at /checkout/success.

## 8. Mobile layouts — how the same design adapts

The generated image includes three mobile mockups along the bottom: the homepage, product listing, and cart.

These are not separate websites. They are responsive versions of the same pages.

### Mobile homepage

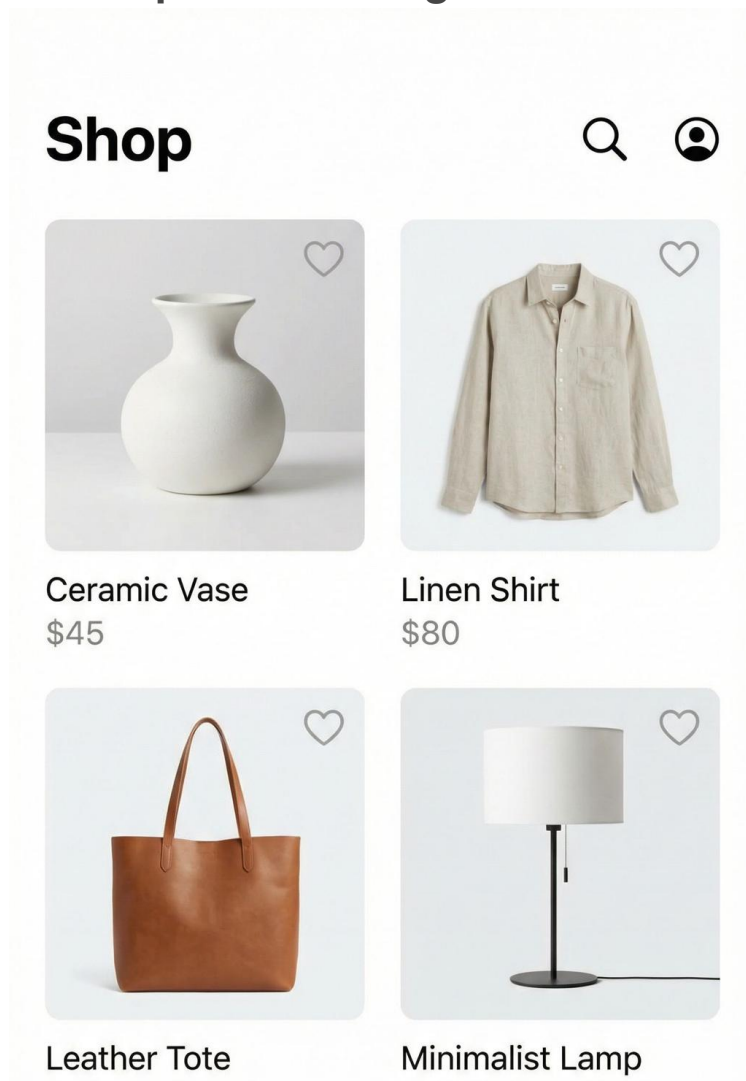


Changes from desktop

- Navigation links collapse into a compact menu.
- Logo and action icons stay visible.
- Hero content stacks vertically.
- Product images remain prominent.
- Collection cards use a narrower grid or horizontal scrolling where appropriate.

Function: Keep brand discovery and shopping accessible on small screens.

## Mobile product listing

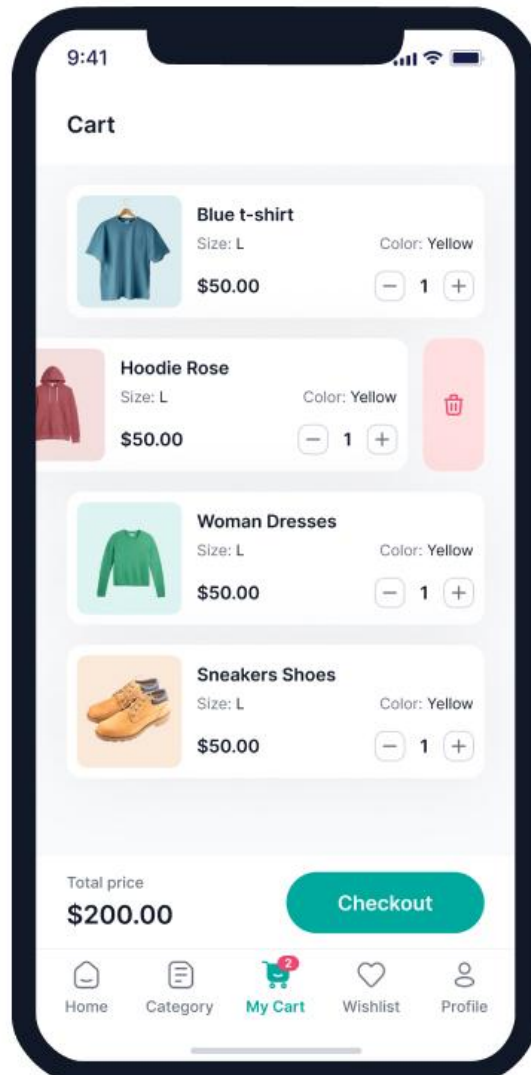


Changes from desktop

- Sidebar filters become a filter drawer or bottom sheet.
- Search and sorting controls fit above the grid.
- Product cards display in two columns when space permits.
- Product names and prices remain readable.

Function: Preserve filtering and product discovery without forcing a desktop layout onto a phone.

## Mobile cart



Changes from desktop

- Cart items are stacked vertically.
- Product thumbnails are smaller but still recognizable.
- Quantity controls remain easy to tap.
- The order summary moves below the items or into a compact sticky summary.
- The checkout button spans the available width.

Function: Let users review their order and proceed without horizontal scrolling.

### Responsive behavior to implement

| Screen width              | Expected layout                                          |
|---------------------------|----------------------------------------------------------|
| Large desktop, 1200px+    | Full navigation and 3–4 product columns                  |
| Tablet, 768–1199px        | Compact navigation and 2–3 columns                       |
| Mobile, below 768px       | Collapsed menu and 2-column product grid                 |
| Small mobile, below 380px | Tighter gaps, smaller thumbnails, carefully wrapped text |

These are suggested breakpoints and layout targets. Test the actual site across widths rather than assuming a layout will work because it matches one screenshot.

## 9. How all pages share the same code

The most important architectural rule is to avoid building every page independently.

### Shared design system

Colors · Typography · Spacing · Buttons · Inputs

### Root layout

Header · Main content · Footer · Cart provider  
Homepage

Product listing

Product detail

Cart & checkout

All routes reuse shared components and a single product data source.

For example, if you change the header's height, logo size, or spacing, that change should appear consistently on every page.

If you update the ProductCard component, the homepage's featured products and the main catalog should both reflect that change.

This is a key frontend engineering skill: reusable components should produce a consistent visual result wherever they appear.

## 10. Map each design element to your codebase

| Visual element               | Component or file            | Responsibility                     |
|------------------------------|------------------------------|------------------------------------|
| Logo and navigation          | layout/Header.tsx            | Consistent navigation              |
| Search and account icons     | layout/MainNav.tsx           | Search and account actions         |
| Hero image and headline      | home/Hero.tsx                | Brand introduction                 |
| Collection cards             | home/FeaturedCollections.tsx | Category navigation                |
| Product cards                | products/ProductCard.tsx     | Reusable product presentation      |
| Filter sidebar               | products/ProductFilters.tsx  | Category, price, and color filters |
| Main product image           | products/ProductGallery.tsx  | Image switching                    |
| Variant selector             | products/ProductInfo.tsx     | Product customization              |
| Cart rows                    | cart/CartItem.tsx            | Quantity and removal               |
| Order total                  | cart/CartSummary.tsx         | Price calculations                 |
| Checkout fields              | checkout form components     | Input and validation               |
| Global colors and typography | app/globals.css              | Visual consistency                 |
| Product catalog              | data/products.ts             | Shared product information         |
| Shared cart state            | context/CartContext.tsx      | Cross-page cart behavior           |

## 11. The implementation order I recommend

Do not attempt to build all the pages at once. Work through the following sequence:

1. Design foundation: Define fonts, colors, spacing, page width, button

styles, and product-image proportions.

2. Shared header: Match the logo, navigation links, search, account, and cart icon before creating the other pages.
3. Homepage: Finish the hero, collections, featured products, brand story, and footer.
4. Product catalog: Reuse the product cards and implement real search, filtering, and sorting.
5. Product details: Implement image selection, variants, quantity, and add-to-cart behavior.
6. Cart and checkout: Implement shared cart state, totals, validation, and simulated order confirmation.
7. Responsive polish: Test all routes at desktop and mobile widths, and correct any inconsistent navigation, spacing, or interactions.

One final distinction: the generated image is a design reference, not a complete specification. Some tiny text, icons, prices, and spacing in the mockup may be approximate. Use the overall layout and visual language as your guide, then make deliberate, consistent decisions in the actual implementation.

For your frontend internship portfolio, the finished website should demonstrate both visual fidelity and working functionality. A consistent header, shared product components, coherent responsive layouts, and reliable interactions will matter more than reproducing every decorative detail in the generated image.



